
FastMem-RMT: Two-Timescale Memory for Continual Associative Retrieval

Adam Suliman

adamsulimanofficial@gmail.com

Abstract

Continual learning in language models is often addressed through replay, regularization, or parameter-efficient adaptation, while architectural memory mechanisms remain comparatively underexplored. We study *Fast Memory Recurrent Memory Transformer* (Fast Memory RMT), a two-timescale recurrent-memory architecture that separates rapidly updated memory tokens from slower parameter updates. Our main evaluation uses continual associative-retrieval tasks derived from multi-query associative recall, where models must repeatedly acquire new key–value associations while retaining earlier ones. Across class-incremental associative-retrieval experiments, Fast Memory RMT consistently improves retention and plasticity relative to base RMT and Transformer attention baselines, suggesting that fast recurrent memory can act as a useful buffer against catastrophic forgetting in controlled NLP-style continual learning. We further include an incremental CIFAR-100 implementation as a secondary proof of concept, showing that the same memory mechanism can be ported beyond synthetic sequence recall. These results position Fast Memory RMT as a promising architecture for continual associative retrieval, while motivating further validation on larger natural-language continual-learning benchmarks.

1 Introduction

Continual learning is becoming increasingly important for language models because deployed systems must adapt to changing knowledge, changing user preferences, new domains, and new tasks without repeatedly retraining from scratch. Recent NLP and LLM continual-learning tutorials emphasize catastrophic forgetting, plasticity loss, data scheduling, and evaluation pipelines as central challenges for modern language systems [Wu et al., 2025, Srijith et al., 2025]. Most practical methods, however, still focus on replay buffers, regularization, adapter-style updates, or parameter-efficient finetuning. In contrast, this work studies whether a memory-centric architecture can directly improve continual retention and plasticity on controlled NLP-style tasks.

We focus on associative retrieval because it is a clean probe of memory behavior in sequence models. Zoology introduced multi-query associative recall (MQAR) to measure recall in efficient language models and argued that synthetic recall tasks can reveal architectural bottlenecks before expensive language-model pretraining [Arora et al., 2023]. Recurrent Memory Transformer (RMT) adds segment-level memory tokens to Transformers and gives the model a persistent channel for carrying information across segments [Bulatov et al., 2022]. More recent associative-memory variants, such as Associative Recurrent Memory Transformer (ARMT), show that recurrent-memory architectures can be effective for very long-context retrieval and benchmarks such as BABILong [Rodkin et al., 2024, Kuratov et al., 2024]. These results motivate a narrower question: can a fast-updated recurrent memory reduce forgetting and preserve plasticity in continual associative retrieval?

We propose and evaluate Fast Memory RMT as a modification of base RMT. The model retains RMT-style segment recurrence but separates the memory into a slow learned initializer and a fast

memory tensor. The fast memory is updated manually after backpropagation, while slow network weights are updated less frequently. This creates a two-timescale learning mechanism: fast memory can absorb stage-local association updates, while the rest of the model changes more conservatively.

Our main contribution is not a broad claim that Fast Memory RMT solves continual NLP. Instead, we make a more precise claim: Fast Memory RMT is promising for *continual associative retrieval*, where the task sequence requires repeated acquisition and retention of key–value associations. The main benchmark is a 5-stage class-incremental associative-retrieval setting implemented in a Zoology fork. We report forgetting, backward transfer, forward transfer, plasticity, and seen-stage accuracy. We also include horizon, interference, and vocab-growth diagnostics, plus a secondary incremental CIFAR-100 proof-of-concept implementation.

The contributions are:

1. We implement Fast Memory RMT and base RMT in a Zoology-style architecture-testing harness for synthetic NLP-style recall.
2. We construct continual associative-retrieval benchmarks that turn MQAR-like recall into stage-wise continual learning tasks.
3. We show that tuned Fast Memory RMT substantially outperforms base RMT and Transformer/MHA on the current class-incremental associative-retrieval benchmark.
4. We document a negative vocab-growth MQAR result, showing that not every synthetic continual recall task is immediately useful as an architecture-ranking benchmark.
5. We provide an incremental CIFAR-100 implementation as a secondary proof of concept for testing the fast-memory idea outside the main NLP-style setting.

2 Related Work

Recurrent memory architectures. RMT augments a Transformer with memory tokens passed between sequence segments, allowing a fixed-size recurrent state to carry information across contexts longer than the local attention window [Bulatov et al., 2022]. ARMT extends this line toward associative storage and very long-context reasoning, reporting strong results on associative retrieval and BABILong-style tasks [Rodkin et al., 2024, Kuratov et al., 2024]. Fast Memory RMT differs from these works in its objective: rather than primarily extending context length, we evaluate whether a two-timescale memory update helps under a continual learning protocol with stage transitions and forgetting metrics.

Synthetic recall and architecture discovery. Zoology argues that synthetic tasks are useful for debugging and comparing language-model architectures before scaling to large pretraining runs [Arora et al., 2023]. MQAR was proposed to better approximate real in-context associative recall than simpler synthetic recall tasks. Our work builds on this philosophy but changes the evaluation axis: instead of testing static recall, we ask whether an architecture can repeatedly acquire new association sets while preserving previous ones.

Long-context evaluation. Benchmarks such as BABILong and RULER evaluate long-context retrieval, aggregation, and reasoning beyond simple needle-in-a-haystack tests [Kuratov et al., 2024, Hsieh et al., 2024]. These benchmarks are natural future targets for Fast Memory RMT. In the present work, however, we deliberately start with smaller, controlled synthetic tasks because the compute budget is limited and because the goal is architectural falsification before large-scale NLP validation.

Continual learning and plasticity. Continual learning studies catastrophic forgetting, forward transfer, backward transfer, and loss of plasticity. Recent LLM-oriented treatments highlight the need for robust evaluation pipelines, task sequencing, and adaptation mechanisms [Wu et al., 2025, Srijith et al., 2025]. The incremental CIFAR-100 repository used here originates from work on plasticity maintenance through weight and unit reinitialization, including comparisons to continual backpropagation and ReDo [Hernandez-Garcia et al., 2025]. We use the CIFAR implementation only as a secondary proof of concept; the central paper claim concerns Fast Memory RMT on continual associative retrieval.

3 Fast Memory RMT

3.1 Base RMT

Base RMT processes a sequence in segments. For each segment, memory tokens are prepended or arranged in a read/write layout, the segment is processed by a Transformer-style block, and the resulting memory state is passed to the next segment. In the implemented fork, the relevant base class is `BaseRMTMixer`. Its forward pass initializes memory, iterates over segments of length `segment_len`, calls a segment processor, optionally detaches memory between segments, records memory diagnostics, and concatenates the segment outputs.

The default layout used in our experiments is a read/write memory layout. Informally, each segment contains read memory, ordinary input tokens, and write memory:

$$[M_{\text{read}}, x_{t:t+L}, M_{\text{write}}].$$

The read memory gives the segment access to past state, while the write memory aggregates information from the current segment and becomes the recurrent state for the next segment. This provides a compact recurrent bottleneck of size

$$S_{\text{state}} = n_{\text{mem}} d_{\text{model}}.$$

3.2 Fast Memory RMT

Fast Memory RMT inherits the segmented computation of base RMT but separates memory into a slow learned initializer and a fast-updated tensor. The implementation replaces the single RMT memory parameter with:

$$M_{\text{slow}} \quad \text{and} \quad M_{\text{fast}}.$$

During training, the effective memory is computed as

$$M_{\text{eff}} = M_{\text{fast}} + M_{\text{slow}} - \text{sg}(M_{\text{slow}}),$$

where $\text{sg}(\cdot)$ denotes stop-gradient. In value, M_{eff} equals the fast memory, but gradients can still shape the slow initializer. After `loss.backward()`, the trainer calls `update_fast_memory()`, optionally clips the fast-memory gradient, applies a manual gradient step

$$M_{\text{fast}} \leftarrow M_{\text{fast}} - \eta_{\text{fast}} \nabla M_{\text{fast}} \mathcal{L},$$

and clears the fast-memory gradient. Slow parameters are updated by the normal optimizer, but only every `rmt_slow_update_freq` batches.

This mechanism creates a two-timescale system. Fast memory can change on every batch, while slow weights and the slow memory initializer change less frequently. The stable configuration used in the main result resets fast memory at epoch boundaries, uses `rmt_fast_lr=0.005`, clips fast-memory gradients at 1.0, and updates slow parameters every four batches.

3.3 Continual Evaluation Metrics

Let $A_{t,i}$ be the accuracy on evaluation stage i after training through stage t . The main metrics are:

$$\text{SeenAvg}(t) = \frac{1}{t+1} \sum_{i=0}^t A_{t,i},$$

$$\text{Forgetting}(i, T) = \max_{k \in \{i, \dots, T\}} A_{k,i} - A_{T,i},$$

$$\text{BWT}(T) = \frac{1}{T} \sum_{i=0}^{T-1} (A_{T,i} - A_{i,i}),$$

and plasticity at stage t is measured by current-stage learning accuracy $A_{t,t}$. Forward transfer is measured by evaluating future stages before training on them and comparing against a random or initial baseline.

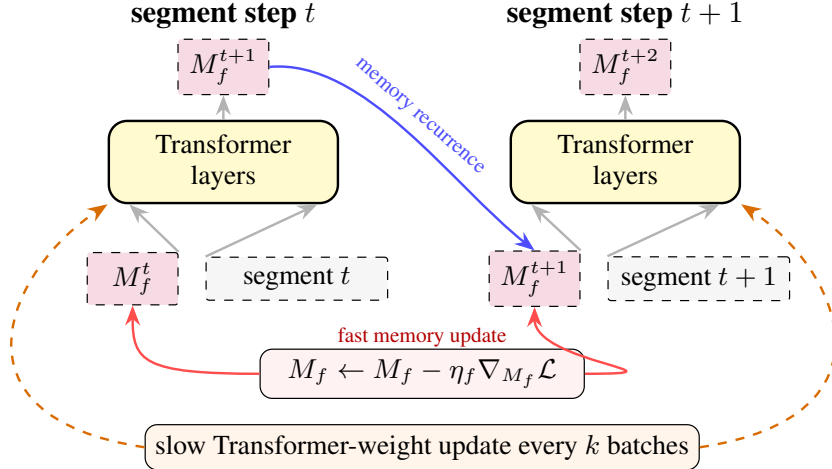


Figure 1: Fast Memory RMT extends standard segment-level RMT recurrence with a two-timescale update. The memory state is passed between segments as in RMT, but it is also adapted by a fast gradient update on the current batch. Transformer parameters are updated more slowly, allowing memory to absorb rapid task-specific changes while preserving slower model weights.

4 Experiments

4.1 Research Questions

We evaluate three questions:

1. Does Fast Memory RMT reduce catastrophic forgetting relative to base RMT and Transformer/MHA?
2. Does the fast-memory mechanism preserve plasticity across sequential association stages?
3. Does fast memory produce useful transfer, or does it merely improve current-stage memorization?

4.2 Main Benchmark: Class-Incremental Associative Retrieval

Our primary benchmark is a class-incremental associative-retrieval task designed to isolate the form of continual learning most directly targeted by Fast Memory RMT. Rather than evaluating the model on broad natural-language tasks immediately, we use a controlled sequence-learning setting in which each stage introduces a new block of key-value associations. The learner is trained only on the current stage, but after each stage it is evaluated on all associations encountered so far. This protocol makes forgetting directly measurable while keeping the underlying skill fixed: the model must repeatedly acquire new associations without losing access to earlier ones.

The task is intentionally closer to multi-query associative recall than to ordinary classification. Each example contains a set of associations and several queries, and the model is scored on whether it retrieves the correct values. Across stages, the association space expands, so good performance requires both current-stage plasticity and retention of previously learned mappings. This makes the benchmark a useful first test for memory architectures: a standard attention model can solve small within-stage recall problems, but continual performance depends on whether the architecture can preserve reusable retrieval behavior while adapting to new association sets.

We evaluate all models under the same staged training protocol and without replay. After each training stage, we compute accuracy on the current stage, accuracy averaged over all seen stages, forgetting on earlier stages, and transfer to held-out future stages when applicable. The main comparison is between a Transformer attention baseline, a base RMT model with recurrent memory tokens, and Fast Memory RMT with a separate fast-updated memory state. Full implementation details, hyperparameters, random seeds, and exact dataset sizes are provided in Appendix ??.

Table 1: Main 5-stage class-incremental associative-retrieval results. Values are mean $\pm$ standard deviation over archived seeds. Higher is better for final cumulative accuracy and seen average accuracy; lower is better for average forgetting.

Model	Final cumulative accuracy	Seen avg. accuracy	Avg. forgetting
Transformer/MHA	0.1237 ± 0.0855	0.0393 ± 0.0247	0.3930 ± 0.0138
Base RMT, $n_{\text{mem}} = 4$	0.7959 ± 0.0263	0.5483 ± 0.0323	0.2042 ± 0.0082
Fast Memory RMT, stable	0.8584 ± 0.0435	0.6620 ± 0.0878	0.1833 ± 0.0638

4.3 Models

We compare three main models:

1. **Transformer/MHA**: a small two-layer attention baseline.
2. **Base RMT**: RMT with $n_{\text{mem}} = 4$ and segment length 64.
3. **Fast Memory RMT**: RMT with $n_{\text{mem}} = 8$, `rmt_fast_lr=0.005`, `rmt_slow_update_freq=4`, `reset_memory_each_epoch=True`, and `rmt_clip_memory_grad=1.0`.

We also discuss a higher-learning-rate Fast Memory RMT tuning variant with `rmt_fast_lr=0.01` and `slow_freq=4`. This variant achieves stronger average performance in the tuning artifact but has larger seed variance, so the lower-learning-rate variant is treated as the stable default.

4.4 Supporting Protocols

Horizon sweep. The horizon sweep uses 5, 10, and 20 stages with fewer examples per stage: 2048 train examples, 512 test examples, and 8 epochs per stage. It is intended to test whether plasticity and retention degrade as the continual-learning sequence becomes longer.

Interference sweep. The interference benchmark tests whether memory helps when associations conflict. It includes no-conflict and repeated-key conditions, with target policies that either require retrieving the latest value or retaining an older value. This separates simple retention from controlled overwrite behavior.

Vocab-growth MQAR diagnostics. The vocab-growth MQAR task uses disjoint key/value token ranges per stage. The diagnostic artifact includes 1-stage solvability, 2-stage forgetting, and 4-stage full-run tests. This benchmark is not used as the headline result because several models remain near random even in the 1-stage solvability test.

5 Results

5.1 Main Class-Incremental Associative-Retrieval Results

Table 1 summarizes the current main result. Fast Memory RMT improves over base RMT in final cumulative accuracy, seen-stage average accuracy, and average forgetting. Transformer/MHA performs poorly in this continual associative-retrieval setting.

The initial Fast Memory RMT default underperformed base RMT, with 0.6848 ± 0.0802 final cumulative accuracy and 0.4191 ± 0.1155 seen-stage average accuracy. This failure motivated the fast-memory tuning sweep. The key empirical change was slowing ordinary parameter updates to every four batches and resetting fast memory at epoch boundaries. With these changes, Fast Memory RMT moves from underperforming base RMT to outperforming it.

5.2 Fast-Memory Tuning

The best mean tuning configuration found so far uses `rmt_fast_lr=0.01` and `slow_freq=4`. It reaches approximately 0.9008 ± 0.0533 final cumulative accuracy, 0.7459 ± 0.1096 seen-stage average accuracy, and 0.1357 ± 0.0472 average forgetting. This is stronger in mean performance than the stable configuration, but its larger variance makes it a less conservative default for the

Table 2: Fast Memory RMT tuning summary. The higher-learning-rate variant is stronger on average but less stable across seeds.

Configuration	Final cumulative accuracy	Seen avg. accuracy	Avg. forgetting
FMRMT, lr = 0.005, slow freq = 4	0.8584 ± 0.0435	0.6620 ± 0.0878	0.1833 ± 0.0638
FMRMT, lr = 0.01, slow freq = 4	0.9008 ± 0.0533	0.7459 ± 0.1096	0.1357 ± 0.0472

Table 3: Vocab-growth MQAR diagnostic result. This is a negative diagnostic, not a headline benchmark.

Model	1-stage final accuracy	4-stage final seen avg. accuracy
Transformer/MHA	0.1171 ± 0.0104	0.0385 ± 0.0494
Base RMT, $n_{\text{mem}} = 4$	0.1294 ± 0.0113	0.0293 ± 0.0360
Fast Memory RMT, stable	0.0313 ± 0.0029	0.0103 ± 0.0044
Random baseline	0.03125	0.03125

report. We therefore use the 0.005 fast-learning-rate version as the main reported Fast Memory RMT configuration and discuss the 0.01 version as an ablation candidate.

5.3 Vocab-Growth MQAR Negative Result

The vocab-growth MQAR diagnostic should not be interpreted as a successful architecture-ranking benchmark. In the 1-stage solvability test, random accuracy is $1/32 = 0.03125$. Base RMT reaches only 0.1294 ± 0.0113 , Transformer/MHA reaches 0.1171 ± 0.0104 , and the stable Fast Memory RMT configuration is essentially random at 0.0313 ± 0.0029 . In the 4-stage full run, final seen average accuracy remains weak for all models, and Fast Memory RMT remains near random. These results suggest that the current stage-local vocab-growth task is not well calibrated: the models often fail to solve the base task, so forgetting measurements mainly reflect task collapse rather than controlled retention.

5.4 Incremental CIFAR-100 Proof of Concept

The incremental CIFAR-100 implementation demonstrates that the fast-memory idea can be ported beyond synthetic NLP-style sequence recall. The repository contains a ViT-parity RMT family with baseline, fast-memory, fast-memory-with-prior, and meta-fast-memory variants. It also includes RMT-specific diagnostics such as memory norm and memory update norm over checkpoints. This implementation is important as a proof of concept for plasticity-loss experiments, especially because preliminary final runs suggest marginal improvement over CBP in plasticity-loss evasion. However, since the core report is about NLP-style continual associative retrieval and because consolidated CIFAR result tables are not yet the strongest artifact, CIFAR should remain in the appendix or a short secondary section rather than the main claim.

6 Discussion

The main result suggests that Fast Memory RMT is useful when the continual-learning task requires repeated acquisition and retention of key-value associations. The improvement is not simply due to adding memory tokens, because base RMT already has memory and is still outperformed by the tuned fast-memory variant. The useful ingredients appear to be: segmented recurrence, a separate fast memory tensor, epoch-level memory reset, and slower ordinary parameter updates.

This supports a two-timescale interpretation. Fast memory can adapt quickly to the current stage, while slow weights are protected from overly frequent updates that may erase earlier associations. The mechanism is especially plausible for associative-retrieval tasks because the recurrent memory state is directly aligned with the operation the task requires: storing and retrieving associations across a sequence and across stages.

The negative vocab-growth MQAR result is also informative. It shows that benchmark design matters. A continual benchmark is only useful for measuring forgetting if the model can first learn

the individual stages. If single-stage solvability is weak, then apparent forgetting or transfer metrics become difficult to interpret. This argues for using class-incremental associative retrieval as the main benchmark and keeping vocab-growth MQAR as a diagnostic failure case until redesigned.

The strongest next external validation would be to adapt Fast Memory RMT to long-context evaluation suites such as RULER and BABILong. These would test whether the synthetic continual associative-retrieval advantage transfers to broader retrieval and reasoning tasks. A later stage should also compare against replay, LoRA-style adaptation, regularization, and other continual-learning baselines in a framework designed for LLM continual learning.

7 Limitations

The main limitation is that the strongest result is synthetic. Although associative retrieval is well motivated by prior work on language-model recall, the current evidence does not yet show that Fast Memory RMT improves full natural-language continual learning. The correct claim is therefore narrow: Fast Memory RMT improves the current continual associative-retrieval benchmark.

Second, the current comparison is primarily architectural. Replay, adapters, LoRA, EWC, GEM, and other continual-learning methods are not yet included in the main benchmark. This makes the result useful for architecture discovery but not sufficient for a broad continual-learning leaderboard claim.

Third, the tuning result shows sensitivity to fast-memory hyperparameters. The higher-learning-rate configuration performs best on average but has higher variance; the stable configuration is more conservative.

8 Conclusion

Fast Memory RMT provides a simple architectural mechanism for two-timescale continual learning in recurrent-memory Transformers. In the current Zoology-based continual associative-retrieval benchmark, it improves over base RMT and Transformer/MHA on retention and seen-stage accuracy. The most defensible current claim is narrow but meaningful: fast-updated recurrent memory is promising for continual associative retrieval and deserves further evaluation on broader long-context and natural-language continual-learning tasks. Incremental CIFAR-100 results provide a secondary proof of concept for plasticity-loss experiments, but the main empirical story should remain focused on NLP-style continual associative retrieval.

References

- Simran Arora, Sabri Eyuboglu, Aman Timalsina, Isys Johnson, Michael Poli, James Zou, Atri Rudra, and Christopher Ré. Zoology: Measuring and improving recall in efficient language models. *arXiv preprint arXiv:2312.04927*, 2023.
- Aydar Bulatov, Yuri Kuratov, and Mikhail S. Burtsev. Recurrent memory transformer. *arXiv preprint arXiv:2207.06881*, 2022.
- J. Fernando Hernandez-Garcia, Shibhansh Dohare, Jun Luo, and Richard S. Sutton. Reinitializing weights vs units for maintaining plasticity in neural networks. In *Proceedings of the 4th Conference on Lifelong Learning Agents*, 2025.
- Cheng-Ping Hsieh, Simeng Sun, Samuel Krizan, Shantanu Acharya, Dima Rekish, Fei Jia, and Boris Ginsburg. Ruler: What’s the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, 2024.
- Yuri Kuratov, Aydar Bulatov, Petr Anokhin, Ivan Rodkin, Dmitry Sorokin, Artyom Sorokin, and Mikhail Burtsev. Babilong: Testing the limits of llms with long context reasoning-in-a-haystack. *arXiv preprint arXiv:2406.10149*, 2024.
- Ivan Rodkin, Yuri Kuratov, Aydar Bulatov, and Mikhail Burtsev. Associative recurrent memory transformer. *arXiv preprint arXiv:2407.04841*, 2024.

- P. K. Srijith, Shrey Satapara, and Sarath Chandar. Continual learning in large language models: Foundations to frontiers. In *Proceedings of the 2025 International Joint Conference on Natural Language Processing: Tutorial Abstracts*. Association for Computational Linguistics, 2025.
- Tongtong Wu, Trang Vu, Linhao Luo, and Gholamreza Haffari. Continual learning of large language models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Tutorial Abstracts*, pages 16–17. Association for Computational Linguistics, 2025.